

// GAME DEVELOPER PORTFOLIO

Independent Game Dev

C++ · SDL2 · OpenGL · Lua · RakNet · TGUI · CEGUI

C++	SDL2	OpenGL	Lua Raw API	RakNet	TGUI	CEGUI
-----	------	--------	-------------	--------	------	-------

Self-taught game developer who built a custom 2D game engine from scratch, built a feature-complete bullet hell game (systems done, content in progress), and is developing a multiplayer turn-based RPG with Touhou characters — all in C++ with Lua scripting.

// 01

CUSTOM 2D GAME ENGINE

Built a complete 2D game engine from scratch in C++. Ships with a map editor, animated object editor, and a Lua scripting layer for rapid development without recompilation.

Core Tech Stack

SDL2 Window management, input, audio

OpenGL GPU rendering, 2D lighting system

TGUI Game UI via custom Lua bindings

CEGUI Editor UI (desktop only)

Lua Gameplay scripting via raw C API

RakNet Multiplayer networking

Rendering Layer System

- User-defined custom rendering layers supported
- GUI layer hardcoded as topmost — prevents z-order bugs by design
- Canvas widget allows OpenGL rendering embedded inside TGUI panels

SteamDeck Compatibility

- Shipped a game on SteamDeck running via Proton
- Independently debugged and resolved CEGUI crash caused by log file permission issues under Proton
- Migrated game client from CEGUI to TGUI to fix crash; kept CEGUI for desktop editor

- Built custom keyboard/controller UI navigation system for mouse-free play

// 02

LUA SCRIPTING & TGUI BINDINGS

Wrote a complete Lua scripting module for TGUI using the raw Lua C API — no third-party wrappers like Sol2 or LuaBridge. Full manual control over the Lua stack.

3-Layer Architecture

```
cpp_*() Raw C++ to Lua bridge – registered via lua_register()

TGUI_*() Lua-side wrapper – one per cpp call, isolated change surface

Class:method() OOP interface used in game scripts
```

Exposed Widgets

- Label, RTLabel (rich text with color/italic tags)
- Button, Panel, ScrollablePanel, EditText
- Picture, Canvas (OpenGL embed), ListView, ComboBox, TabContainer
- YAGNI approach — widgets added on demand, not all upfront

Key Design Decisions

- Raw Lua C API chosen for simplicity and full stack control
- Lua TGUI bindings save C++ recompile time during UI development
- Prompt/dialog system built entirely in Lua — reusable across all screens
- Rich text Tag system defined globally for consistent stat color styling

// 03

KEYBOARD / CONTROLLER NAVIGATION SYSTEM

Built a custom focus navigation system from scratch since TGUI provides no keyboard or controller navigation out of the box. Primary motivation: SteamDeck players should navigate all menus without a mouse.

FocusPanel (C++)

- Manages an ordered list of focusable Label widgets per panel
- std::vector for ordered navigation, std::set for O(1) duplicate checking

- getNextLabel / getPreviousLabel — circular wrap-around in both directions
- getType() bitmask — distinguishes Panel-based vs Gui-based focus contexts

Focus Priority Vector

- NOT a true FILO stack — focused panel moves to front, others keep relative order
- Allows persistent panels to stay registered when focus shifts away
- m_savedFocusPanel maps Panel pointers to their FocusPanel objects
- Lua-side: ControlHandler_reciever_push / remove for panel registration

// 04

TOUHOU-LIKE SHMUP (IN DEVELOPMENT)

A bullet hell / danmaku game with all core systems fully implemented and playable. Pending: art assets, stages, and content pipeline. Engine, mechanics, lighting, and spellcard systems are complete.

2D Lighting System

- Dark/illuminate zones built on top of OpenGL rendering pipeline
- Light radius falloff — dark overlay with alpha-based radial transparency
- Darkness as a gameplay mechanic — bullets from dark zones are harder to read
- Adds intentional difficulty layer unique to this game

Spellcard Swap System

- Multiple spellcards per character — swappable mid-fight
- Gaze resource cost per swap — generated by grazing bullets
- Per-card cooldown system — prevents spam, rewards timing
- Adds tactical resource management layer on top of raw bullet dodging
- Players must manage: bullet patterns AND spellcard economy simultaneously

// 05

TOUHOU TURN-BASED RPG (IN DEVELOPMENT)

A multiplayer turn-based tactical RPG using Touhou Project characters. Inspired by Darkest Dungeon but with an original 3x3 grid formation system.

3x3 Grid Formation System

- Each player places up to 4 characters in a 3x3 grid (9 cells)
- Melee attacks blocked by front-row enemies — positional blocking mechanic
- Ranged attacks reach any row; AOE hits target cell + adjacent cells
- Cell position determines available skills per character
- Empty cells enable Summon and Invisibility mechanics

Turn Order & Speed System

`effectiveSpeed = baseSpeed + agilityBonus + itemBuffs + diceRoll(1~6)`

- Dice roll adds unpredictability — prevents deterministic mirrors
- Speed ties resolved by additional reroll
- Full turn queue broadcast to both players each round for transparency

// CHARACTER PASSIVES

Reimu Hakurei

// passive

At turn start: (15% + missing HP%) chance for a random buff. Gets more dangerous as HP drops — built-in comeback mechanic.

Patchouli Knowledge

// passive

Manashield: absorbs 80% of incoming damage as mana loss instead of HP. Counter-strategy: mana drain.

Hong Meiling

// passive

Stance System: unique per-character mechanic boosting physical attack or defense. Stance switch is a free action.

Yukari Yakumo

// passive

Fate Reflection: 17% chance to reflect incoming attacks to a random cell on either field — including her own side.

Remilia Scarlet

// passive

Fate Manipulation: on failed dodge or missed attack, gets a second roll at reduced chance. Abuses every RNG layer in the game.

// SERVER ARCHITECTURE

Main Server

- Authentication and login system
- Client data storage: unlocked characters, saved formations
- Formation data: which characters occupy which 3x3 cells

Battle Server

- Lobby creation, 2-player matchmaking, formation selection + ready system
- Full disconnect handling — notifies remaining player, session cleanup
- Battle Session object tracks both formations, turn queue, round state
- Server-side validation of all actions — authoritative server model

// CURRENT STATUS

// COMPLETED

- Main server (auth, client data, formations)
- Battle server (lobby, matchmaking, disconnect)
- Client (login, chars, formation editor, lobby)
- Offline prototype (grid, turns, stats, demo skill)
- Agility-based turn order with dice variance
- TGUI Lua bindings module
- Keyboard/controller navigation system

// IN PROGRESS

- PvP battle system (main event)
- Passive skill implementation
- AOE + adjacency logic
- Summon mechanic
- Invisibility mechanic

SKILLS SUMMARY

Languages	C++, Lua (raw C API)
Graphics	OpenGL, SDL2, 2D lighting, layer rendering
UI Frameworks	TGUI, CEGUI, custom Lua widget bindings
Networking	RakNet, authoritative server design, lobby systems
Engine Systems	Custom 2D engine, map editor, animation editor
Game Design	Shmup, turn-based RPG, grid tactics, resource systems
Platform	SteamDeck / Proton, cross-platform debugging
Scripting	Lua scripting layer, hot-reload-style UI development
Engineering	YAGNI, layered architecture, documented codebase